KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

STRING UTILITY BOX

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
KANISHKA A	25BIT121	B.SC IT 'B' (2025 - 2028)
KISHORE S	25BIT122	B.SC IT 'B' (2025 - 2028)
LOHITASREE SR	25BIT123	B.SC IT 'B' (2025 - 2028)
MITHRA M	25BIT124	B.SC IT 'B' (2025 - 2028)
MITHUL KIRUTHIK RS	25BIT125	B.SC IT 'B' (2025 - 2028)

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled **"STRING UTILITY BOX**" submitted in partial fulfilment of the requirements for the course **C PROGRAMMING** is a **record of our original work** carried out under the guidance of **VASANTHKUMAR M**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
KANISHKA A	25BIT121	B.SC IT 'B' (2025 - 2028)	
KISHORE S	25BIT122	B.SC IT 'B' (2025 - 2028)	
LOHITASREE SR	25BIT123	B.SC IT 'B' (2025 - 2028)	
MITHRA M	25BIT124	B.SC IT 'B' (2025 - 2028)	
MITHUL KIRUTHIK RS	25BIT125	B.SC IT 'B' (2025 - 2028)	

Place: COIMBATORE

Date: _____

CERTIFICATE

This is to certify that the project entitled **"STRING UTILITY BOX"** is a **bona fide work** carried out by the following students of **I YEAR B.Sc INFORMATION TECHNOLOGY**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **C PROGRAMMING**.

This project has been completed under my supervision and guidance during the academic period **NOVEMBER 2025**

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	KANISHKA A	25BIT121
2.	KISHORE S	25BIT122
3.	LOHITASREE SR	25BIT123
4.	MITHRA M	25BIT124
5.	MITHUL KIRUTHIK RS	25BIT125

Guide:
(Technical Trainer, KGM)

Head of the Department
(Technical Head, KGM)

Place: COIMBATORE

Date:26/03/2026

ABSTRACT

The project "STRING UTILITY BOX" is developed using the C programming language to perform various string manipulation operations efficiently. The main objective of this project is to provide a collection of useful string functions in a single program, making string handling simpler and more organized.

The system allows users to perform multiple operations such as string length calculation, string concatenation, string comparison, string reversal, and case conversion. Users can select the desired operation from a menu-driven interface and input the required strings. The program then processes the input and displays accurate results instantly.

This project demonstrates the effective use of arrays, functions, and pointers in C to handle and manipulate strings. The modular design ensures clarity, reusability, and ease of understanding. Each operation is implemented as a separate function to maintain structured programming practices.

The design focuses on simplicity, efficiency, and user interaction, making it suitable for beginners to understand string operations in C. Through this project, important concepts such as control flow, function usage, and data handling have been applied successfully.

Overall, this project enhances programming skills and provides practical exposure to string processing techniques, bridging the gap between theoretical knowledge and real-world application in software development.

.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	

8	**References**	

1. INTRODUCTION

1.1 Objective

- To demonstrate the use of strings in C programming.
- To implement common string operations such as:

Finding the length of a string

Copying one string to another

Comparing two strings

Reversing a string

Concatenating (joining) two strings

- To use built-in string functions like strlen(), strcpy(), strcmp(), and strcat().
- To practice control structures like switch-case for menu-based execution.
- To enhance understanding of arrays and user input handling in C.

1.2 Overview

- **Simple and user-friendly menu system**
- **Demonstrates multiple string operations in one program**
- **Uses arrays to store strings**
- **Combines both library functions and user-defined logic**

1.3 Features

- **Menu-driven program for easy user interaction**
- **Performs multiple string operations in one application**
- **Uses built-in string functions like strlen(), strcpy(), strcmp(), strcat()**
- **Implements switch-case control structure for operation selection**
- **Accepts user input dynamically during execution**

2. PROGRAM ANALYSIS

2.1 Problem Definition

Design and implement a menu-driven C program that performs various basic operations on strings. The program should allow the user to input one or two strings and choose an operation such as :

- **finding length.**
- **copying the string.**
- **comparing two strings.**
- **reversing the given string.**
- **concatenating two strings.**

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	**Concepts Used: Arrays, strings, standard library functions (string.h), user input/output, and switch-case control structure.**
I/O Type	Console-based
Optional	File Handling for saving receipts

2.3 Scopes and Limitation

Scope:

- **Performs essential string operations like length, copy, compare, reverse, and concatenate**
- **Helps in understanding basic string handling and functions in C.**
  Limitations :
- **Accepts only single-word input and has fixed string size.**
- **Lacks input validation and continuous menu execution (no loop).**

3. SYSTEM DESIGN

3.1 Modules

The program consists of four main modules: the input module for accepting user strings, the menu module for displaying options and taking user choice, the processing module for performing string operations like length, copy, compare, reverse, and concatenate, and the output module for displaying the results.

Module-Detailed Description

Module Name	Description
Input Module	Responsible for accepting input from the user. It reads one or two strings using input functions and stores them in character arrays for further processing.
Menu Module	Displays the list of available string operations and accepts the user's choice. It ensures proper navigation by directing the program to the selected operation using control structures.
Processing Module	Performs the core logic of the program. It executes string operations such as finding length, copying, comparing, reversing, and concatenating using built-in functions and user-defined logic.
Output Module	Displays the results of the selected operation in a clear and formatted manner, providing feedback to the user based on their input and choice.

Module Interaction Overview

1. The Input Module accepts string(s) from the user

2. The Menu Module displays options and takes user choice

3. Based on the choice, control is passed to the **Processing Module**

4. The Processing Module performs the selected string operation

5. The Output Module displays the result to the user

4. METHODOLOGY

The methodology of the program involves displaying a menu of string operations, accepting the user's choice, reading the required string(s), processing the input using appropriate string functions or logic, and finally displaying the result to the user.

4.1. Problem Analysis

The problem involves creating a program that can handle and manipulate strings based on user requirements. It requires understanding how strings are stored as character arrays in C and how different operations like length calculation, copying, comparison, reversing, and concatenation can be performed using built-in functions and logic. The program must efficiently take user input, process it according to the selected operation, and produce accurate results while maintaining a simple and user-friendly structure.

4.2. System Design

The system is designed as a **simple modular and menu-driven application**. It consists of four main components: **Input, Menu, Processing, and Output modules**. The user interacts with the system through the menu interface, where they select a desired string operation. The input module collects the required string data, which is then passed to the processing module. The processing module executes the selected operation using built-in string functions or custom logic. Finally, the output module displays the result to the user. The overall flow follows a **sequential and controlled structure using switch-case**, ensuring that each operation is performed based on the user's choice in an organized manner.

4.3. Program Development

The program is developed using the **C programming language** to implement a menu-driven string utility system. Initially, required header files like `stdio.h` and `string.h` are

included to support input/output operations and string functions. Character arrays are declared to store user input. A menu is designed to display different string operations, and the user's choice is taken using input functions.

Based on the selected option, a **switch-case structure** is used to execute the corresponding string operation such as length calculation, copying, comparison, reversing, or concatenation. Standard library functions and simple logic are used to process the strings. Finally, the result is displayed to the user, ensuring a clear and structured flow of execution.

4.4. Testing and Validation

The program is tested by providing different input values for each menu option to ensure that all string operations work correctly. Various test cases, such as normal strings, empty inputs, and different combinations of strings, are used to verify accuracy. Validation is performed by checking whether the output matches the expected results for each operation. Any errors or unexpected behavior are identified and corrected to ensure the program runs reliably and produces correct results.

4.5. Result and Documentation

The program successfully performs various string operations such as finding length, copying, comparing, reversing, and concatenating strings based on the user's choice. The output is displayed correctly for different test cases, confirming that the program works as intended.

The documentation includes details about the program's objective, methodology, modules, system design, and testing process. It provides a clear explanation of how the program is developed and executed, making it easy to understand, use, and maintain.

4.6. Algorithm

An **algorithm** is a **step-by-step logical procedure** used to solve a specific problem or perform a computation.

In programming, it defines the exact sequence of actions the computer must follow to achieve the desired output from a given set of inputs. For string utility box,it perform different string operations such as length calculation, copying, comparison, reversing, and concatenation based on the user's choice.

Purpose of the Algorithm

The purpose of the algorithm is to provide a clear and systematic step-by-step procedure for performing different string operations based on the user's choice, ensuring accurate processing and organized execution of the program.

Algorithm in the Project

The algorithm used in this project follows a **menu-driven approach**, where the program first displays a list of string operations and accepts the user's choice.

1. **Start the program**
2. Display the menu with available string operations (length, copy, compare, reverse, concatenate)
3. Prompt the user to enter their choice
4. Read the user's choice using input function
5. Use a **switch-case statement** to handle different options
6. **If choice = 1 (Find Length):**
   - Accept a string from the user
   - Use `strlen()` to calculate length
   - Display the length
7. **If choice = 2 (Copy String):**
   - Accept a string from the user
   - Use `strcpy()` to copy it into another string
   - Display both original and copied strings
8. **If choice = 3 (Compare Strings):**
   - Accept two strings from the user
   - Use `strcmp()` to compare them
   - Display whether they are equal or not
9. **If choice = 4 (Reverse String):**
   - Accept a string from the user
   - Reverse the string using logic or function
   - Display the reversed string
10. **If choice = 5 (Concatenate Strings):**

- Accept two strings from the user

- Use `strcat()` to join them
- Display the concatenated string

11. **If choice is invalid:**

- Display “Invalid choice”

12. **End the program**

Importance of Algorithm in the Project

- Provides a clear step-by-step solution to the problem
- Ensures accurate and efficient execution of operations
- Makes the program easy to understand and implement
- Helps in debugging and maintaining the program easily

4.7. Flowchart

A flowchart is a graphical representation of an algorithm that shows the sequence of steps involved in solving a problem. It uses symbols like **oval, rectangle, diamond, and arrows** to represent the start/end, processes, decisions, and flow of control. In this program, the flowchart helps to visually understand how the menu-driven string operations are executed step by step based on the user’s choice.

Explanation of Flow

- **Start the program**
- **Display the menu of string operations**
- **Accept the user’s choice**
- **Take required string input(s)**
- **Perform the selected operation**
- **Display the result**
- **Show error message for invalid choice (if any)**
- **End the program**

5. SOURCE CODE

```
#include<stdio.h>
#include<string.h>
void main(){
```

```
char str1[128],str2[128];
printf("__String utility box__\n");
printf("1.Find length\n");
printf("2.Copy the string\n");
printf("3.Comparing two strings\n");
printf("4.Reverse the string\n");
printf("5.joining two strings\n");
int choice;
printf("Enter your choice:");
scanf("%d",&choice);
switch(choice){
        case 1:
                printf("Enter the word:");
                scanf("%s",&str1);
                printf("length:%d",strlen(str1));
                break;
        case 2:
                printf("Enter the word:");
                scanf("%s",&str1);
                strcpy(str2,str1);
                printf("original string:%s\n",str1);
                printf("copied string:%s\n",str2);
                break;
        case 3:
                printf("Enter the first word:");
                scanf("%s",&str1);
                printf("Enter the second word:");
                scanf("%s",&str2);
                if(strcmp(str1,str2)==0){
```

```
                printf("the given words are same");
            }
            else{
                printf("the given words are not same ");
            }
            break;
        case 4:
            printf("Enter the word:");
            scanf("%s",&str1);
            strrev(str1);
            printf("reversed word:%s",str1);
            break;
        case 5:
            printf("Enter the first word:");
            scanf("%s",&str1);
            printf("Enter the second word:");
            scanf("%s",&str2);
            printf("concatenated word:%s",strcat(str1,str2));
            break;
    default:
      printf("Invalid choice");
    }
}
```

SOURCE CODE EXPLANATION:

1) Headers & Macros

`#include<stdio.h>`

This header file is used for **input and output operations** such as `printf()` and `casnf()`.

`#include<string.h>`

This header file provides **string handling functions** like `strlen()`, `strcpy()`, `strcmp()`, and `strcat()`.

---

2) Data Models (Structures)

- The program uses **character arrays (`char str1[128], str2[128]`)** to store strings, which act as the primary data structures.
- No user-defined structures (`struct`) are used; instead, simple arrays are sufficient for handling string operations in this program.

---

3) Main Function — Variables & Menu Setup

- The `main()` function is the entry point of the program where execution begins.
- It declares variables such as **character arrays (`str1, str2`)** to store input strings and an **integer (`choice`)** to store the user's selection.
- The menu is displayed using multiple `printf()` statements, showing different string operation options.
- The program then uses `scanf()` to read the user's choice, which determines the operation to be performed using a switch-case structure.

---

4) Print Title & Menu

- The program first displays a **title** ("String Utility Box") using `printf()` to indicate the purpose of the program.
- It then prints a **menu of options** (length, copy, compare, reverse, concatenate) using multiple `printf()` statements.
- Each option is clearly numbered so the user can easily choose an operation.
- This section provides a **user-friendly interface** for interacting with the program.

---

5) Print any string operation

- After performing the selected string operation, the program uses `printf()` to display the result.
- The output is printed in a clear format, such as showing the length, copied string, comparison result, reversed string, or concatenated string.
- This ensures the user can easily understand the result of the chosen operation.

---

6) Program Exit

- Normal termination of main().

Control Flow Summary

1. Start the program execution
2. Display the title and menu options
3. Read the user's choice
4. Check the choice using switch-case
5. Accept required input string(s)
6. Execute the selected string operation
7. Display the result using output statements
8. Handle invalid choice (if any)
9. End the program

6. INPUT AND OUTPUT SCREENS

Input Screen 1:

__String Utility Box__

1. Find length

2. Copy the string

3. Comparing two strings

4. Reverse the string

5. Joining two strings

Enter your choice: 1

Enter the word: hello

Input Screen 2:

__String Utility Box__

1. Find length

2. Copy the string

3. Comparing two strings

4. Reverse the string

5. Joining two strings

Enter your choice: 2

Enter the word: world

Output Screen:

1) Length Operation Output

Length: 5

2) Copy Operation Output

Original string: world

Copied string: world

3)Comparing two strings

Enter your choice: 3

Enter the first word: hello

Enter the second word: hello

The given words are same

4) Reverse Operation Output

Reversed word: olleh

5) Concatenation Operation Output

Concatenated word: helloworld

7. DISCUSSION & CONCLUSION

7.1 Discussion

This program demonstrates a simple and effective way of performing basic string operations using a **menu-driven approach** in C. It helps in understanding how strings are handled using character arrays and standard library functions. The use of a switch-case structure makes the program organized and easy to follow.

However, the program has some limitations, such as handling only single-word inputs and lacking continuous execution. Despite this, it serves as a good foundation for beginners to learn string manipulation and can be further improved by adding more features and validations.

7.2 Conclusion

The program successfully implements a **menu-driven string utility system** in C to perform basic string operations like length, copy, compare, reverse, and concatenation. It demonstrates the use of string functions, control structures, and user input handling effectively. Overall, the program is simple, user-friendly, and useful for understanding fundamental concepts of string manipulation in C programming.

8. REFERENCES

- Kernighan & Ritchie, *The C Programming Language*, Prentice Hall, 1988.
- TutorialsPoint – C Programming
- GeeksforGeeks – Restaurant Billing in C
- Dev-C++ User Manual